Mobile Application Development (MAP711S)

Language Fundamentals

Simon H. Muchinenyika

Language Fundamentals

Table of Contents

- Kotlin Introduction
  - literals
  - if and when expressions, ranges, Elvis operator
- Functions
  - lambda expression
- Classes & Objects
  - Constructors, Inheritance, Abstraction, Polymorphism
  - Screen

Language Fundamentals

Kotlin

- open-source programming language, maintained by JetBrains, & first released 2011.
- named after Kotlin Island near St. Petersburg, Russia.
- considered a general-purpose language for many platforms, including JVM, Android, JS, and Native.
- Kotlin's syntax is similar to Java but has numerous differences, including:
  - extension features
  - type inference
- Kotlin official site: https://kotlinlang.org/

Language Fundamentals

Kotlin

- Android apps officially developed using Kotlin, a statically typed language;

```
fun main(args: Array<String>) {
    println("Hello, world!”)
}
```

```
fun main() {
    println("Hello, world!”)
}
```

```
fun main() = println("Hello, world!")
```

- all three pieces of code are the same & correct.

Language Fundamentals

Kotlin Features

- Kotlin supports multiple paradigms, such as:
  - imperative programming.
  - object-oriented programming.
  - generic programming.
  - functional programming.
- Also supports tools such as:
  - anonymous functions.
  - higher-order functions.

Language Fundamentals

Variable

- Variables: val, var, const val:

```
var width: Int = 12          //mutable
val length = 25              //immutable, type is inferred
const val NAME = ”GRACE”     //can be calculated at compile time
width = 15                   //ok
var height: Int              //type required if not initialised
height = 5                   //deferred assignment
width = 15                   //ok
length = 25                  //error
```

- once a type is assigned, it cannot be changed.

Language Fundamentals

Functions

```
fun sum(a: Int, b: Int): Int {
  return a + b
}
```

```
fun mul(a: Int, b: Int) = a * b
```

- single expression function;

```
fun printMul(a: Int, b: Int): Unit {
  println(mul(a, b))
}
```

```
fun printMul1(a: Int = 1, b: Int) {
  println(mul(a, b))
}
```

```
fun printMul2(a: Int, b: Int = 1) = println(mul(a, b))
```

- Unit means a function does not return anything and can be omitted;
- arguments can have default values.

Language Fundamentals

if expression

```
fun maxOf(a: Int, b: Int): Int {
  if (a > b) {
  return a
  } else {
  return b
  }
}
```

```
fun maxOf(a: Int, b: Int) =
    if (a > b) {
      a
    } else {
      b
    }
```

```
fun maxOf(a: Int, b: Int) = if (a > b) a else b
```

Language Fundamentals

When expression

```
when (x) {
    1 -> print("x == 1")
    2 -> print("x == 2")
    else -> {
        print("x is neither 1 nor 2")
    }
}
```

when returns, the same way that if does.

```
when {
    x < 0 -> print("x < 1")
    x > 0 -> print("x > 2")
    else -> {
        print("x == 0”)
    }
}
```

The condition can be inside of the branches

Language Fundamentals

When statement

```
fun serveTeaTo(customer: Customer) {
  val teaSack = takeRandomTeaSack()

    when (teaSack) {
        is OolongSack -> error("We don't serve Chinese tea like $teaSack!")
        in trialTeaSacks, teaSackBoughtLastNight ->
            error("Are you insane?! We cannot serve uncertified tea!")
    }

    teaPackage.brew().serveTo(customer)
}
```

when can accept several options in one branch. else branch can be omitted if when block is used as a *statement*.

Language Fundamentals

Ranges

```
val numberOfStudents = 50
if (numberOfStudents in 1..100) {
    println(numberOfStudents)
}
```

What is the output?

NB: The range operator does not contain spaces.

Language Fundamentals

Null Safety

```
val notNullText: String = "Definitely not null"
val nullableText1: String? = "Might be null"
val nullableText2: String? = null

fun funny(text: String?) {
  if (text != null)
  println(text)
  else
  println("Nothing to print :(")
}

fun funnier(text: String?) {
  val toPrint = text ?: "Nothing to print :("
  println(toPrint)
}
```

Language Fundamentals

Elvis Operator ? :

```
fun loadInfoById(id: String): String? {
  val item = findItem(id) ?: return null
  return item.loadInfo() ?: throw Exception("...")
}
```

- if the expression on the left is not null, the Elvis operator returns it;
- otherwise it returns the expression on the right;
- NB: the expression on the right is only evaluated if the left-hand side is null.

Language Fundamentals

Safe Calls

- useful in chains;

```
fun printDepartmentHead(employee: Employee) {
  println(employee.department?.head?.name)
}
```

- to print only for non-null values, use the safe call operator together with let;

```
employee.department?.head?.name?.let { println(it) }
```

Language Fundamentals

Lambda Expressions

```
val sum: (Int, Int) -> Int = { x: Int, y: Int -> x + y }
val mul = { x: Int, y: Int -> x * y }
```

According to Kotlin convention, if the last parameter of a function is a function,
then a lambda expression passed as the corresponding argument can be placed outside the parentheses:

```
val badProduct = items.fold(1, { acc, e -> acc * e })

val goodProduct = items.fold(1) { acc, e -> acc * e }
```

If the lambda is the only argument, the parentheses can be omitted entirely (the documentation calls this feature "trailing lambda as a parameter"):

```
run({ println("Not Cool") })
run { println("Very Cool") }
```

Language Fundamentals

Kotlin Classes

```
class Person(val name: String, val surname: String, private var age: Int) {

    init {
        findJob()
    }

    constructor(name: String, parent: Person) : this(name, parent.surname, 0)
}
```

- primary constructor can be ommitted if brackets are empty;
- the order of initialisation is primary constructor = > init block = > secondary constructor;
- class properties are declared in the primary, & not the secondary constructor.

Language Fundamentals

Constructors

```
open class Point(val x: Int, val y: Int) {
    constructor(other: Point) : this(other.x, other.y) { ... }

    constructor(circle: Circle) : this(circle.centre) { ... }
}
```

- constructors can be chained, but should always call the primary constructor in the end;
- inheritor class must call the parent's constructor:

```
class ColoredPoint(val color: Color, x: Int, y: Int) : Point(x, y) { ... }
```

Language Fundamentals

init blocks

```
class Example(val value: Int, info: String) {
  val anotherValue: Int
  var info = "Description: $info"

  init {
    this.info += ", with value $value"
  }

  val thirdValue = computeAnotherValue() * 2

  private fun computeAnotherValue() = value * 10

  init {
    anotherValue = computeAnotherValue()
  }
}
```

- a class may contain several init blocks;
- blocks are “joined” into one, preserving the declaration order when executing.
- constructor parameters are accessible in init blocks, making it possible to use this.

Language Fundamentals

Abstraction

```
interface RegularCat {
    fun pet()
    fun feed(food: Food)
}

interface SickCat {
    fun checkStomach()
    fun giveMedicine(pill: Pill)
}
```

```
abstract class RegularCat {
    abstract val name: String

    abstract fun pet()
    abstract fun feed(food: Food)
}

abstract class SickCat {
    abstract val location: String

    abstract fun checkStomach()
    fun giveMedicine(pill: Pill) {}
}
```

- interfaces cannot have a state;
- abstract classes cannot have an instance but can have a state.

Language Fundamentals

Encapsulation

```
abstract class RegularCat {
    protected abstract val isHungry: Boolean
    private fun poop(): Poop { /* do the thing */ }
    abstract fun feed(food: Food)
}
class MyCat : RegularCat() {
    override val isHungry: Boolean = false
    override fun feed(food: Food) {
        if (isHungry) { /* do the thing */ }
        else { poop() } // MyCat cannot poop
    }
}
```

- four modifiers used: private, protected, iternal, public.
- Why MyCat cannot poop?

Language Fundamentals

Inheritance

```
class SickDomesticCat : RegularCat(), CatAtHospital {
    override var isHungry: Boolean = false
        get() = field
        set(value) {...}

    override fun pet() {...}

    override fun feed(food: Food) {...}

    override fun checkStomach() {...}

    override fun giveMedicine(pill: Pill) {...}
}
```

the keyword open is used to extend non-abstract classes;

you can only inherit from one class and from many interfaces as you wish;

when inheriting from its class, you call its constructor in the same way secondary constructors call the primary.

Language Fundamentals

Inheritance Example

```
abstract class Cat {
  /* final */ fun anotherDay() {
    // various cat activities
    digest(findFood())
    poop(findWhereToPoop())
  }
  private fun poop(where: Place): Poop {...}
  private fun digest(food: Food) {
    // don't know how they work
    poop(findWhereToPoop())
  }
  abstract fun feed(food: Food)
  abstract fun findWhereToPoop(): Place
  abstract fun findFood(): Food
}
```

```
class DomesticCat(
  val tray: Tray,
  val bowl: Bowl
) : Cat() {
  override fun feed(food: Food) {
    // place some food in the bowl
  }

  override fun findWhereToPoop() = tray
  override fun findFood() {
   return bowl.getFood() ?: run {
     // find food somewhere else
   }
  }
}
```

Language Fundamentals

Polymorphism

```
interface DomesticAnimal {
    fun pet()
}

class Dog: DomesticAnimal {
    override fun pet() {...}
}

class Cat: DomesticAnimal {
    override fun pet() {...}
}

fun main() {
    val homeZoo = listOf<DomesticAnimal>(Dog(), Cat())
    homeZoo.forEach { it.pet() }
}
```

```
class PositiveAttitude(startingAttitude: Int) {
    var attitude = max(0, startingAttitude)
        set(value) =
        if (value >= 0) {
            field = value
        } else {
            println("Only positive attitude!")
            field = 0
        }

    var hiddenAttitude: Int = startingAttitude
        private set
        get() {
            if (isSecretelyNegative) {
                println("Don't ask this!")
                field += 10
            }
            return field
        }

    val isSecretelyNegative: Boolean
        get() = hiddenAttitude < 0
```

Language Fundamentals

Properties

- properties optionally have an initialiser, getter & setter;
- field keyword is used to access the fields insides the getter & setter.

Language Fundamentals

Properties continued...

```
open class OpenBase(open val value: Int)

interface AnotherExample {
  /* abstract */ val anotherValue: OpenBase
}

open class OpenChild(value: Int) : OpenBase(value), AnotherExample {
  override var value: Int = 1000
    get() = field - 7
  override val anotherValue: OpenBase = OpenBase(value)
}

open class AnotherChild(value: Int) : OpenChild(value) {
  final override var value: Int = value
    get() = super.value // default get() is used otherwise
    set(value) { field = value * 2 }
  final override val anotherValue: OpenChild = OpenChild(value) // Notice that we use OpenChild here, not
OpenBase
```

- properties may be open or abstract;
- interfaces can have properties, but they are always abstract.
- you can prohibit further overriding by marking a property final.

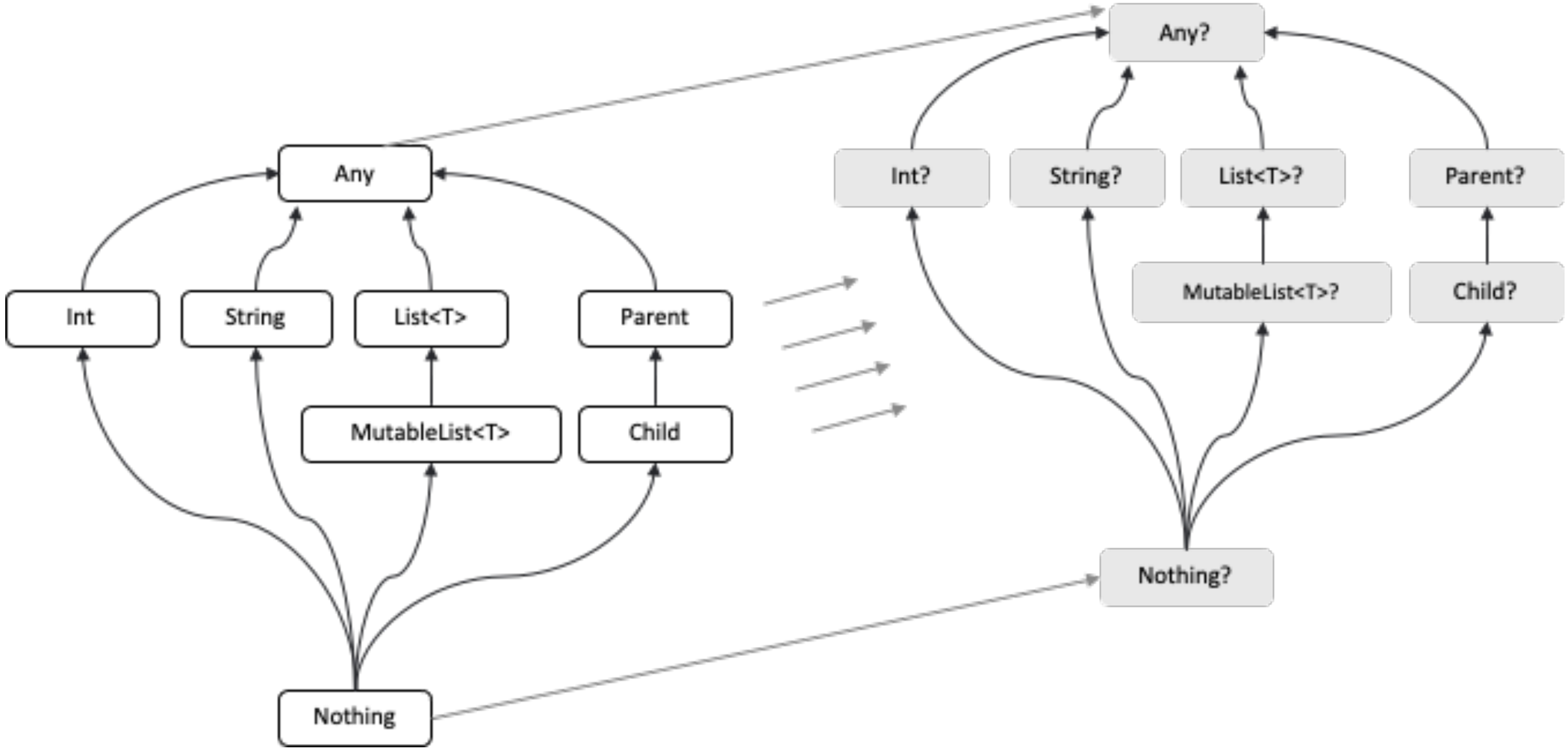
Language Fundamentals

Data Classes

- by just adding the keyword data, the compiler automatically derives:
  - equals( ) and hashCode( )
  - toString( )
  - componentN( ) functions
  - copy( )

Language Fundamentals

Kotlin Type Hierarchy



Language Fundamentals

Further Study

- Kotlin Evolution: https://github.com/Kotlin/KEEP
- do you efficiently develop for mobile platforms?
  - WebP Images: https://developers.google.com/speed/webp/docs/using
  - Accessible A

Language Fundamentals

References

- Inheritance:

  https://kotlinlang.org/docs/inheritance.html

- Init Blocks & Constrctor: https://stackoverflow.com/questions/55356837/what-is-the-difference-between-init-block-and-constructor-in-kotlin

- Object Oriented Programming:

  https://en.wikipedia.org/wiki/Object-oriented_programming

- Getters & Setters:

  https://www.baeldung.com/kotlin/getters-setters

Language Fundamentals

Thank You

Questions?

T: +264 61 207 2054

F: +264 61 207 9054

E: smuchinenyika@nust.na